

Linguagem C e sistemas embarcados



Funções em C



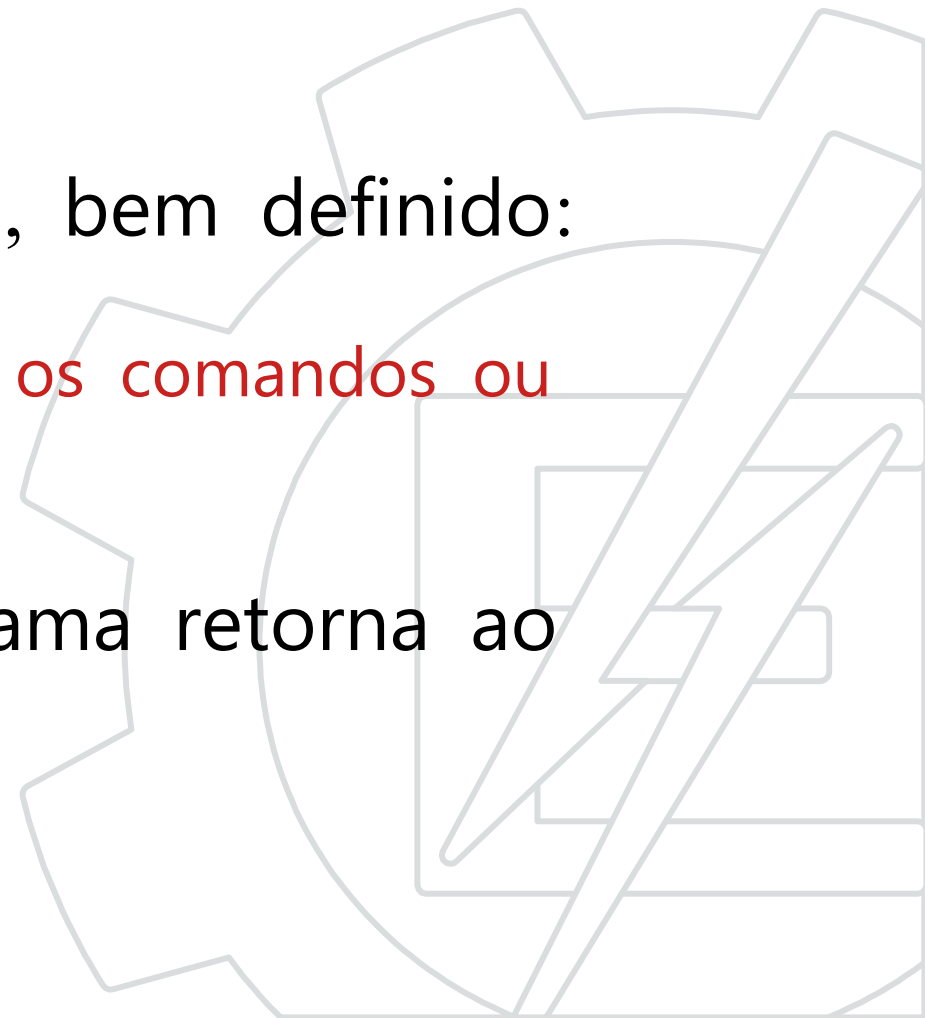
Funções em C

- Um programa em C, obrigatoriamente possui uma função chamada `main()`.
- Existem muitas funções pré-definidas.
exemplo: `strcmp()`, `strcpy()`, ...
A diretiva `#include` possibilita o uso dessas funções.
- O programador pode criar suas funções, `seno()`, `coseno()`, `imprimir()`...



Funções em C

- Uma função é um conjunto de comandos para realizar uma tarefa específica.
- O ciclo de vida da função é, em geral, bem definido:
 - Inicia-se com a chamada da função.
 - Finaliza-se ao terminar de executar todos os comandos ou até encontrar um comando return.
- Depois de finalizada a função o programa retorna ao ponto em que a função foi chamada.



Funções em C

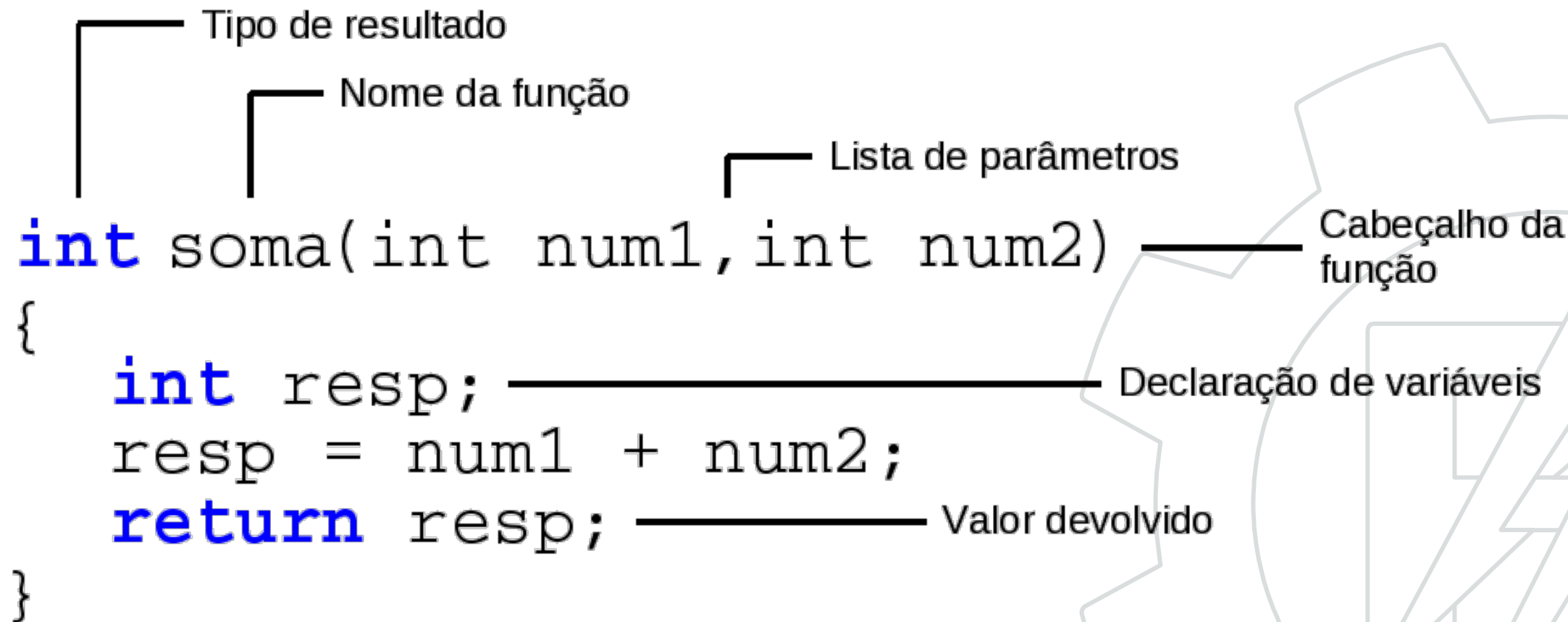
- Uma função é definida da seguinte forma:

```
tipo nome(Parametros){  
    //corpo da funcao  
    return expressao;  
}
```

- **tipo**: valor devolvido pela função.
- **nome**: identificador da função.
- **Parâmetros**: variáveis passadas.
 - Os parâmetros são separados por vírgula.
- **expressão**: valor devolvido pela função.



Funções em C



The diagram illustrates the components of a C function definition. It shows the following code with annotations:

```
int soma(int num1, int num2)
{
    int resp;
    resp = num1 + num2;
    return resp;
}
```

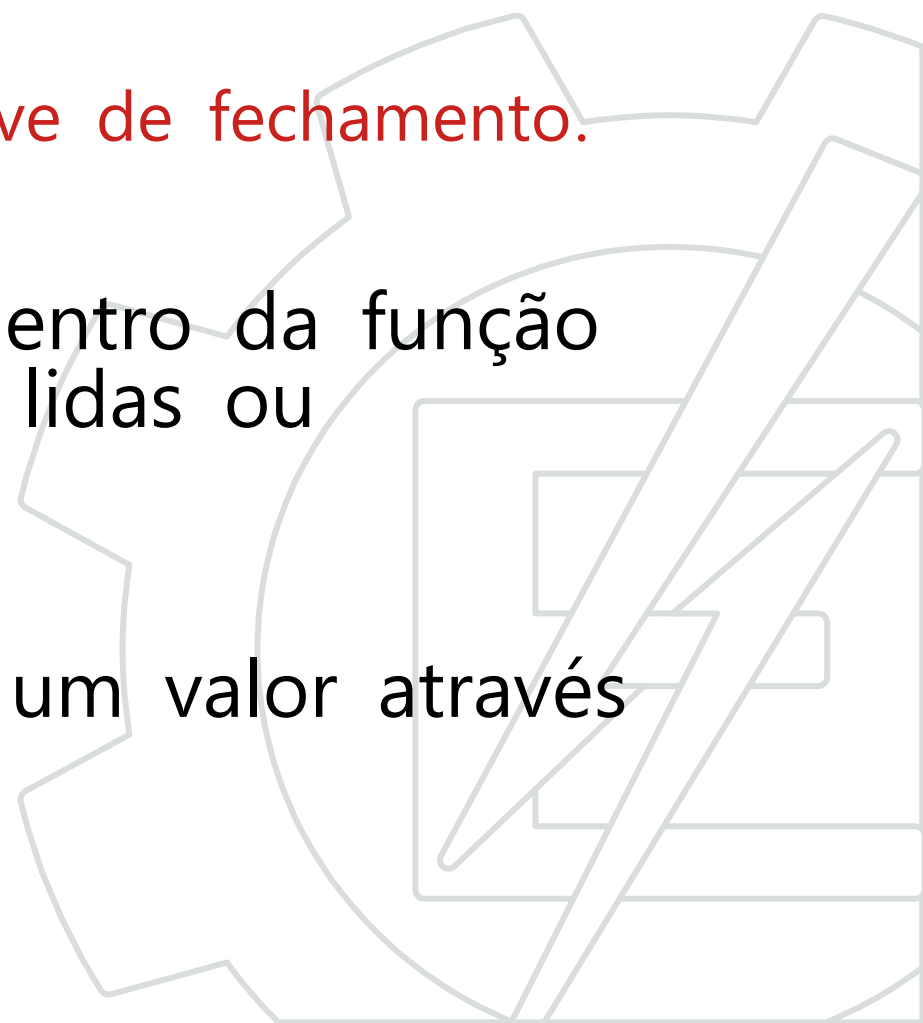
Annotations and their corresponding parts of the code:

- Tipo de resultado** (Type of result): Points to the `int` at the beginning of the function signature.
- Nome da função** (Function name): Points to the `soma` identifier.
- Lista de parâmetros** (List of parameters): Points to the parentheses containing `int num1, int num2`.
- Cabeçalho da função** (Function header): Points to the entire line `int soma(int num1, int num2)`.
- Declaração de variáveis** (Variable declaration): Points to the line `int resp;` inside the function body.
- Valor devolvido** (Returned value): Points to the `resp` variable in the `return resp;` statement.

A large, faint gear icon is visible in the background on the right side of the slide.

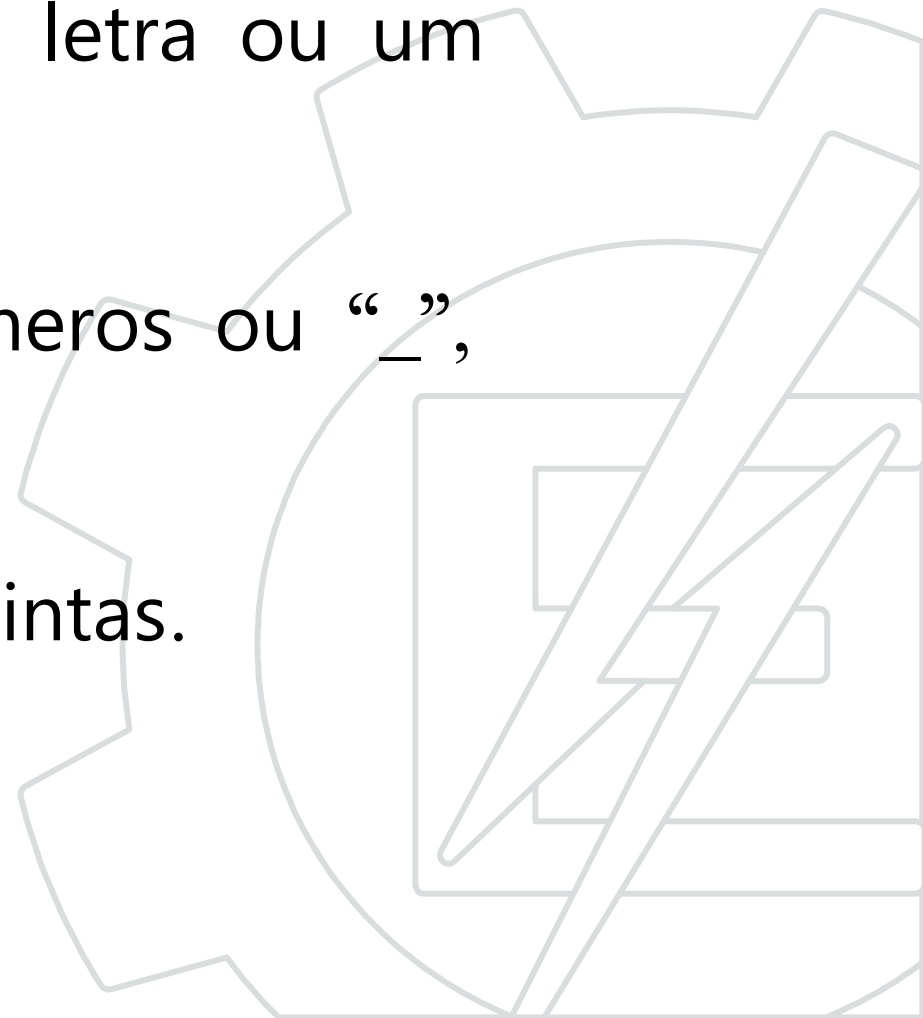
Funções em C

- Corpo da função: tem que estar entre o abre chave "{" e o fecha chave "}".
 - Não há “ponto e vírgula” depois da chave de fechamento.
- As constantes e variáveis declaradas dentro da função são locais à função e não podem ser lidas ou acessadas fora da função.
- A função só pode devolver (retornar) um valor através do “return”.



Nome da Função

- Um nome de função começa com uma letra ou um caracter “_”.
- O nome pode conter tantas letras, números ou “_”, quanto desejar o programador.
- Letras maiúsculas e minúsculas são distintas.



Retorno de funções em C

- O tipo de dado retornado deve ser um dos 5 tipos de C ou um tipo definido no programa.
- O tipo “void” serve para indicar que a função não retorna nenhum valor.

```
//retorna um valor inteiro
int max(int x, int y){...}

//retorna um double
double media(double x1, double x2){...}

//retorna um float
float soma(int numElem){...}

//não retorna nada
void tela(void){...}
```

Tipos de dados em C: char, int, float, double e void.

Exemplo

- Função para imprimir 10 vezes o caracter 'A'

```
void desenha(void){  
    int i; //variavel local  
    for(i=0; i<10; i++){  
        lcdChar('A'); //função lcdChar de uma biblioteca  
    }  
}  
  
void main(void){  
    desenha(); //chama a função desenha  
}
```



Protótipo de Funções

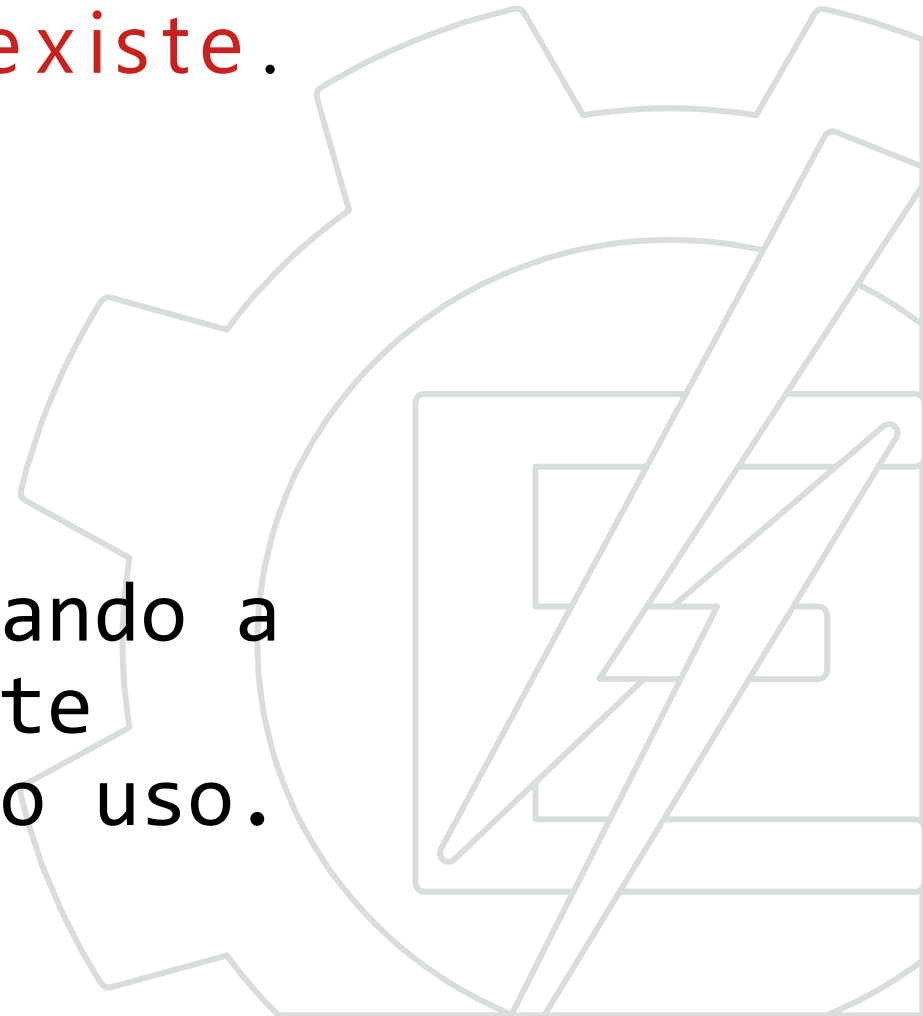
- É necessário declarar ou definir a função **antes do seu uso**.
- A declaração sem a implementação é chamada de protótipo da função.
- O protótipo possui somente o cabeçalho da função, terminado com ponto e vírgula.

```
tipo nome(Parâmetros);
```



Protótipo de Funções

- Um protótipo **declara simplesmente que a função existe.**
- Os protótipos devem ser declarados logo no início.
- O protótipo pode ser omitido quando a função for definida completamente (com todos os comandos) antes do uso.



Exemplos

```
void desenha(void){
    int i;
    for(i=0; i<10; i++){
        lcdChar('A');
    }
}

void main(void){
    desenha();
}
```

```
void desenha(void); ←
void main(void){
    desenha();
}

void desenha(void){
    int i;
    for(i=0; i<10; i++){
        lcdChar('A');
    }
}
```

Passagem de parâmetros

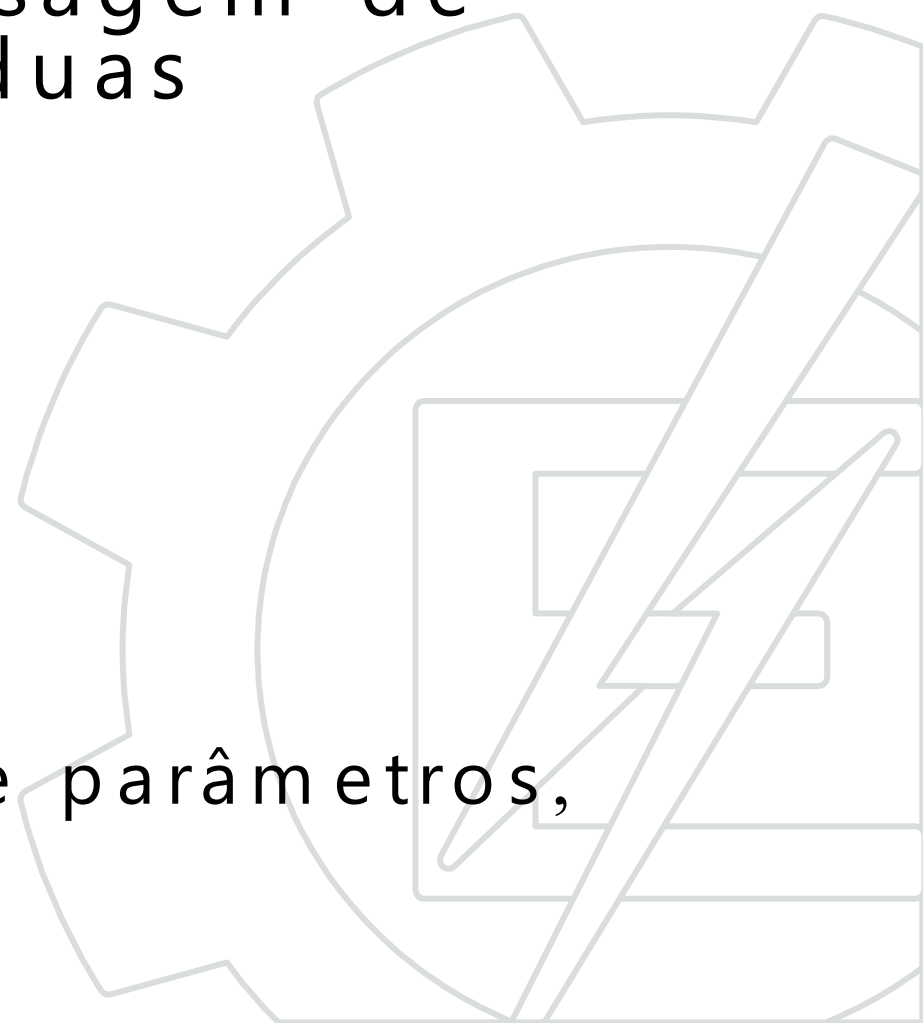


Passagem de parâmetros

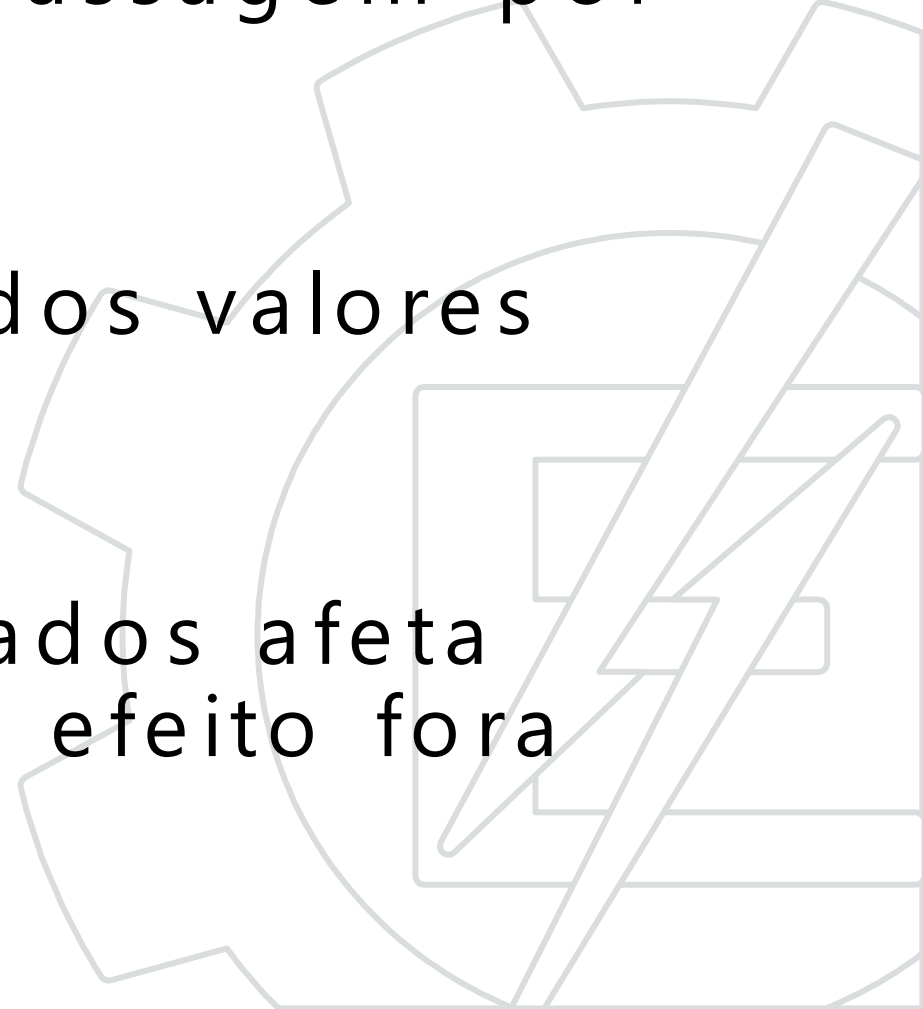
A linguagem C permite a passagem de parâmetros para funções de duas maneiras:

- Por valor.
- Por referência.

Obs: Quando a função não recebe parâmetros, “void” é utilizado.



Passagem de parâmetros por valor

- É também conhecida como passagem por cópia.
 - A função recebe uma cópia dos valores passados.
 - A mudança dos valores passados afeta somente a função e não tem efeito fora dela.
- 

Passagem de parâmetros por valor

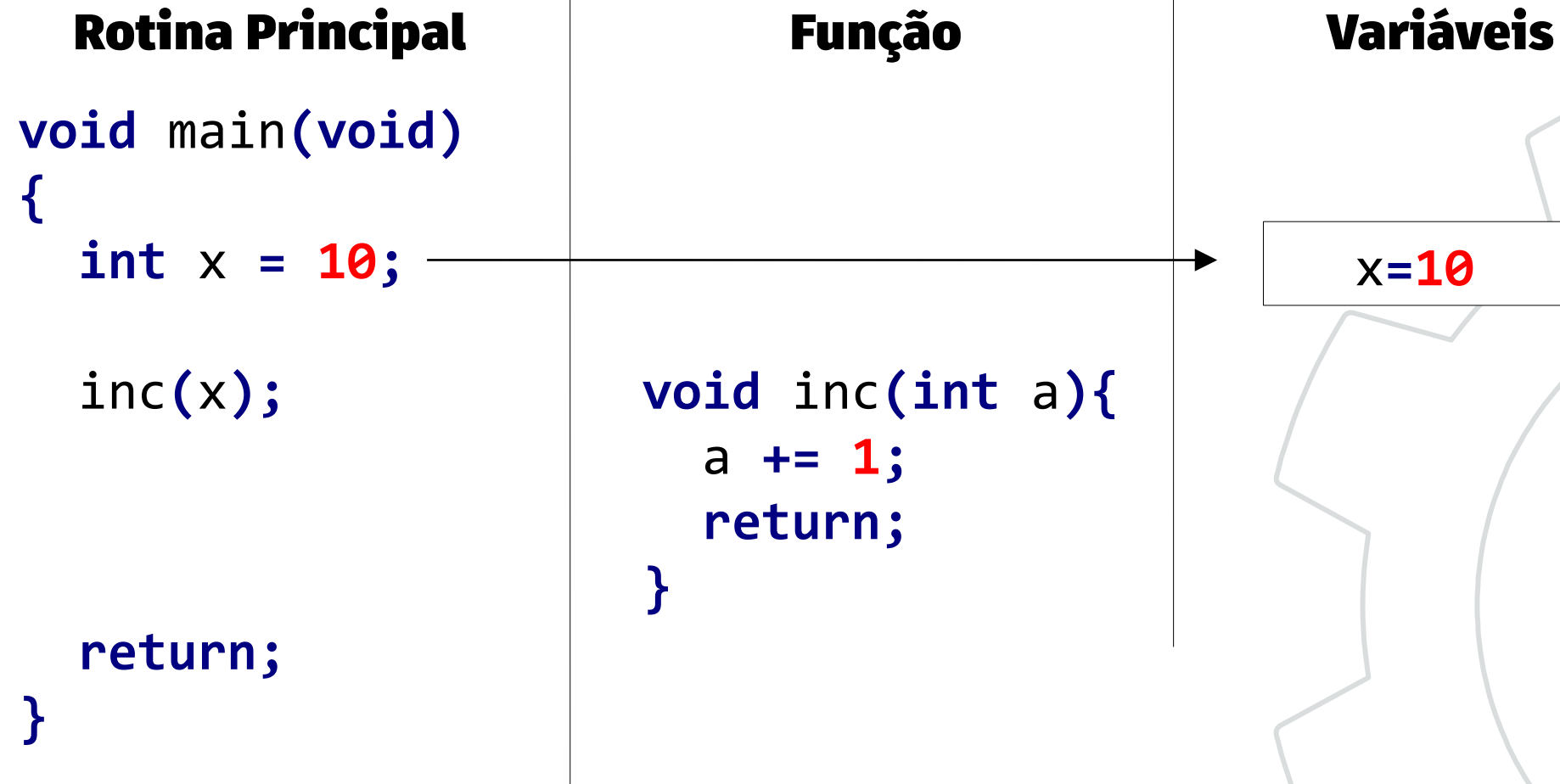
Exemplo:

```
void incrementa(int a){  
    a += 1;  
    return; //Não é obrigatório return neste caso  
}
```

```
void main(void){  
    int x = 10;  
    incrementa(x);  
    return;  
}
```



Passagem de parâmetros por valor



Passagem de parâmetros por valor

Rotina Principal

```
void main(void)
{
    int x = 10;

    inc(x);

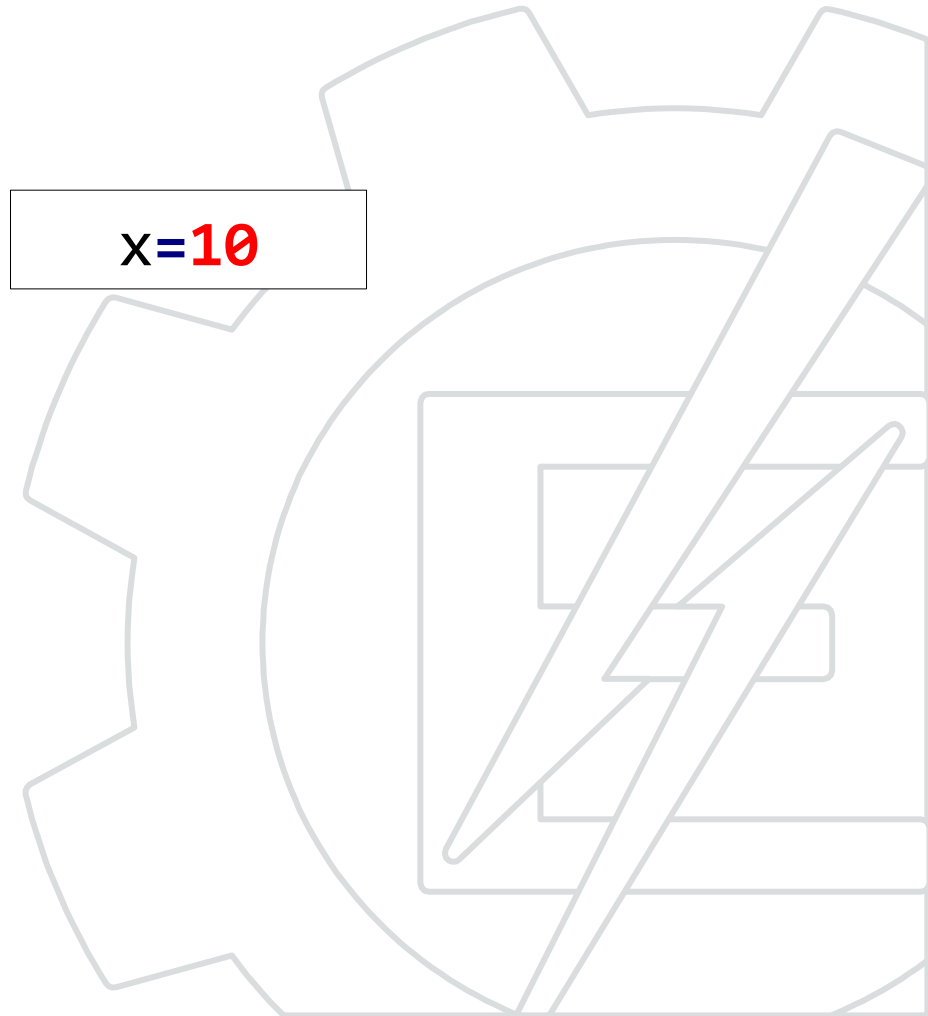
    return;
}
```

Função

```
void inc(int a){
    a += 1;
    return;
}
```

Variáveis

x=10



Passagem de parâmetros por valor

Rotina Principal

```
void main(void)
{
    int x = 10;

    inc(x);

    return;
}
```

Função

```
void inc(int a){
    a += 1;
    return;
}
```

Variáveis

x=10
a=10



Passagem de parâmetros por valor

Rotina Principal

```
void main(void)
{
    int x = 10;

    inc(x);

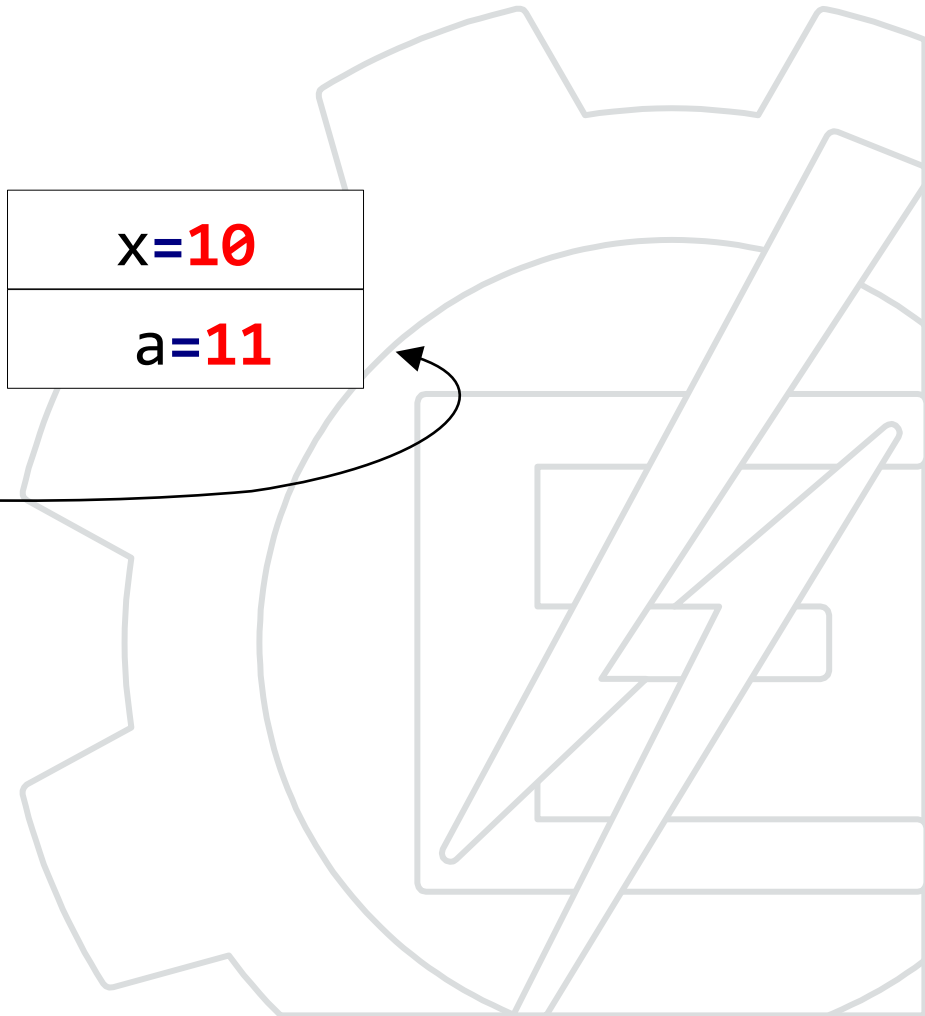
    return;
}
```

Função

```
void inc(int a){
    a += 1;
    return;
}
```

Variáveis

x=10
a=11



Passagem de parâmetros por valor

Rotina Principal

```
void main(void)
{
    int x = 10;

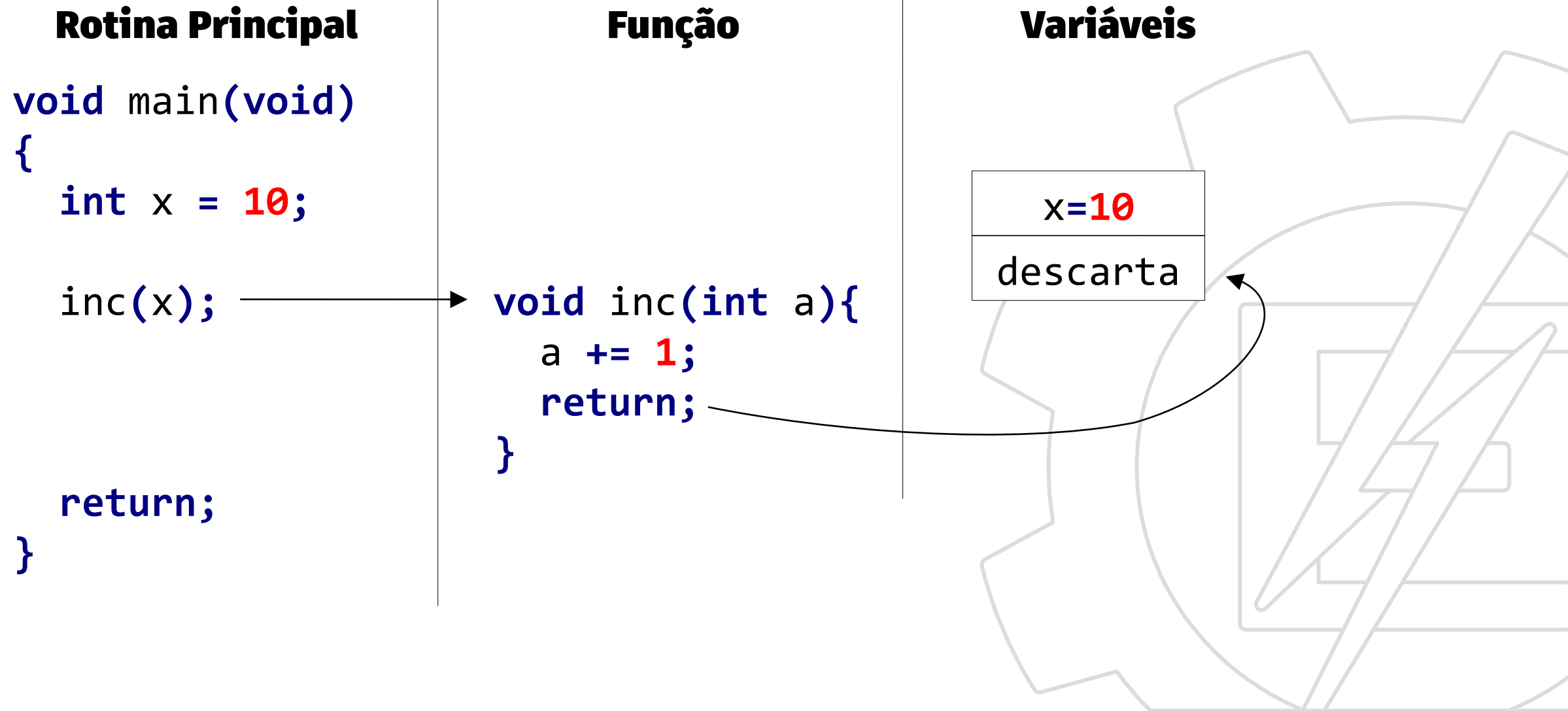
    inc(x);

    return;
}
```

Função

```
void inc(int a){
    a += 1;
    return;
}
```

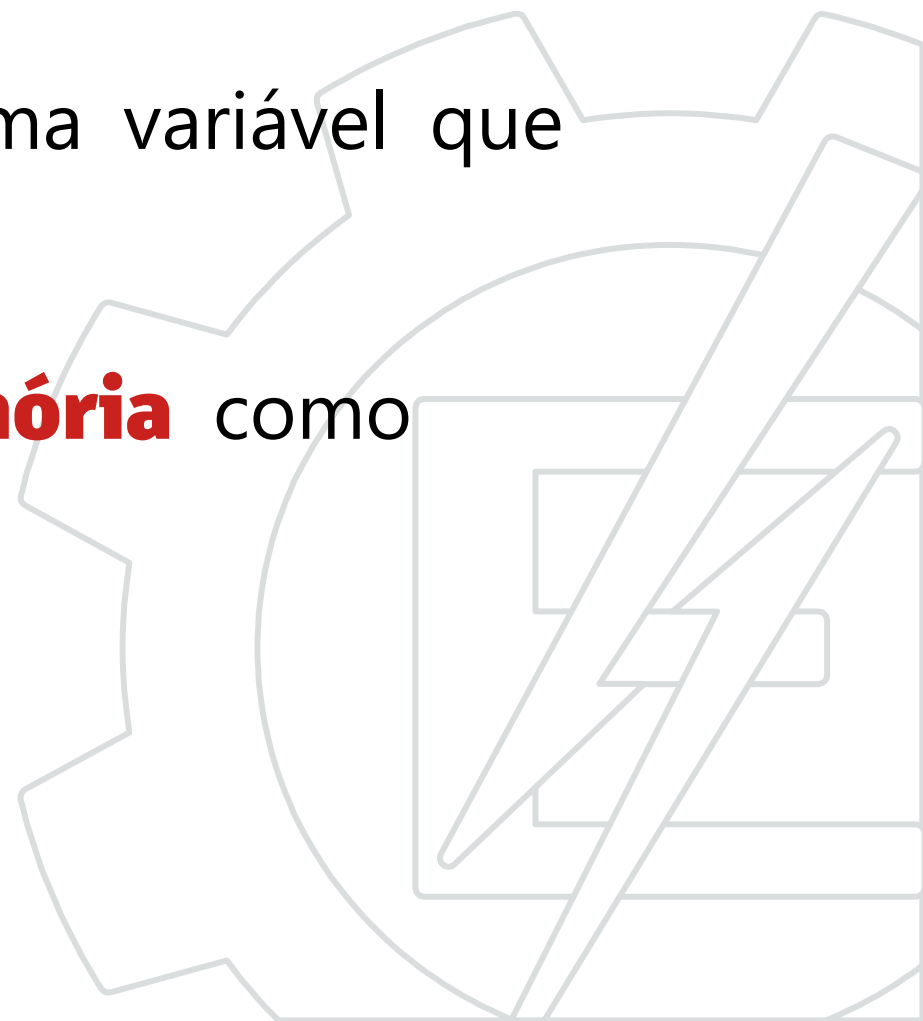
Variáveis



x=10
descarta

Passagem de parâmetros por referência

- A função pode modificar o valor de uma variável que não foi declarada dentro da função.
- A função **recebe um endereço de memória** como parâmetro.



Passagem de parâmetros por referência

- Esse modo permite que uma função “retorne” mais de um parâmetro por vez.

Exemplo:

```
//d e r são dados de saída
void div(int a, int b, int* d, int* r){
    (*d) = a / b;
    (*r) = a % b;
    return;
}

void main(void){
    int d, r;
    div(10, 3, &d, &r);
    //d = 3 e r = 1;
}
```



Passagem de parâmetros por referência

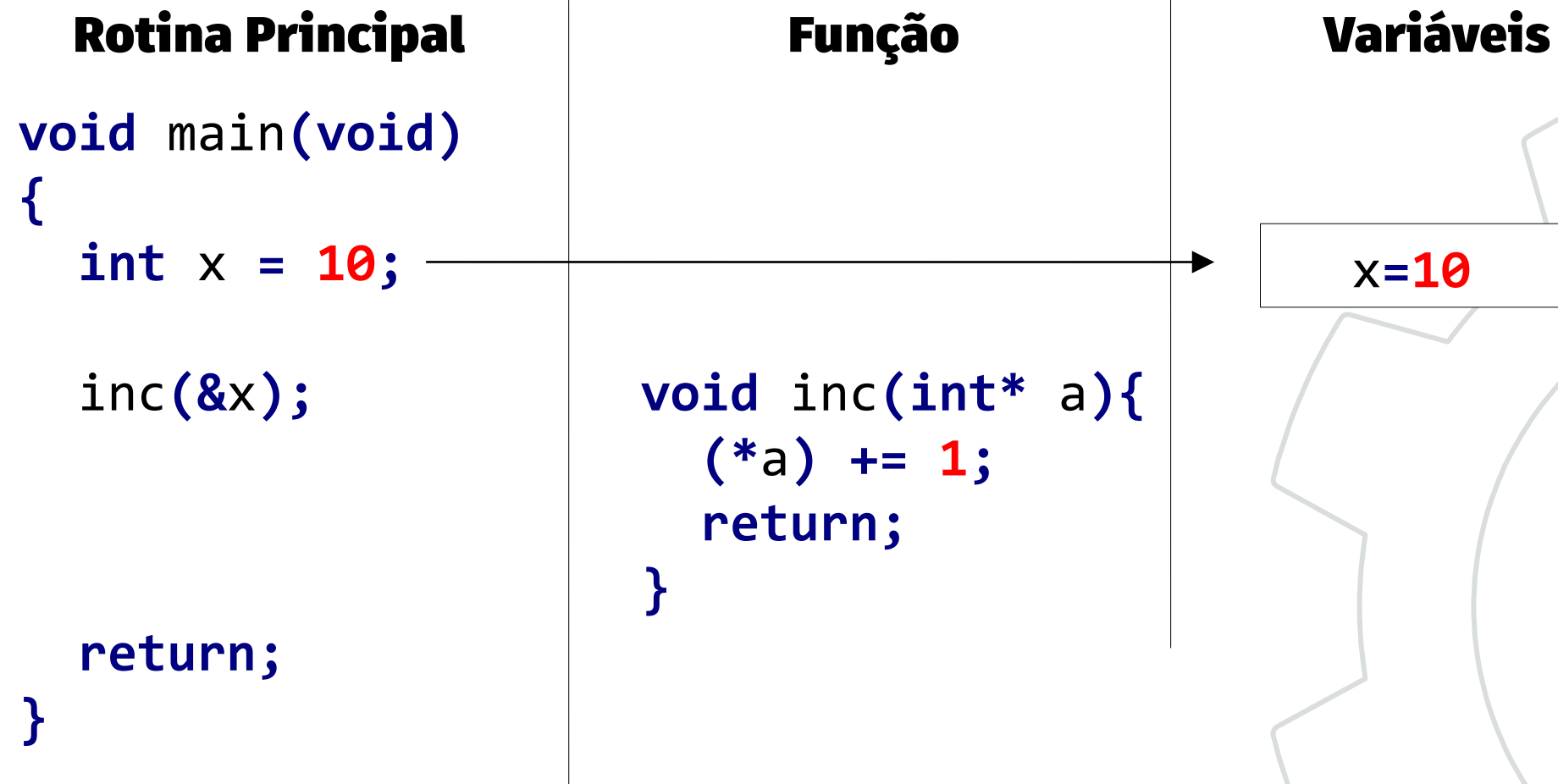
Exemplo:

```
void inc(int *a){  
    (*a)++; // incrementa o conteúdo no endereço a  
    return;  
}
```

```
void main(void){  
    int x = 10;  
    inc(&x);  
    // aqui x = 11  
    ...  
}
```



Passagem de parâmetros por referência



Passagem de parâmetros por referência

Rotina Principal

```
void main(void)
{
    int x = 10;

    inc(&x);

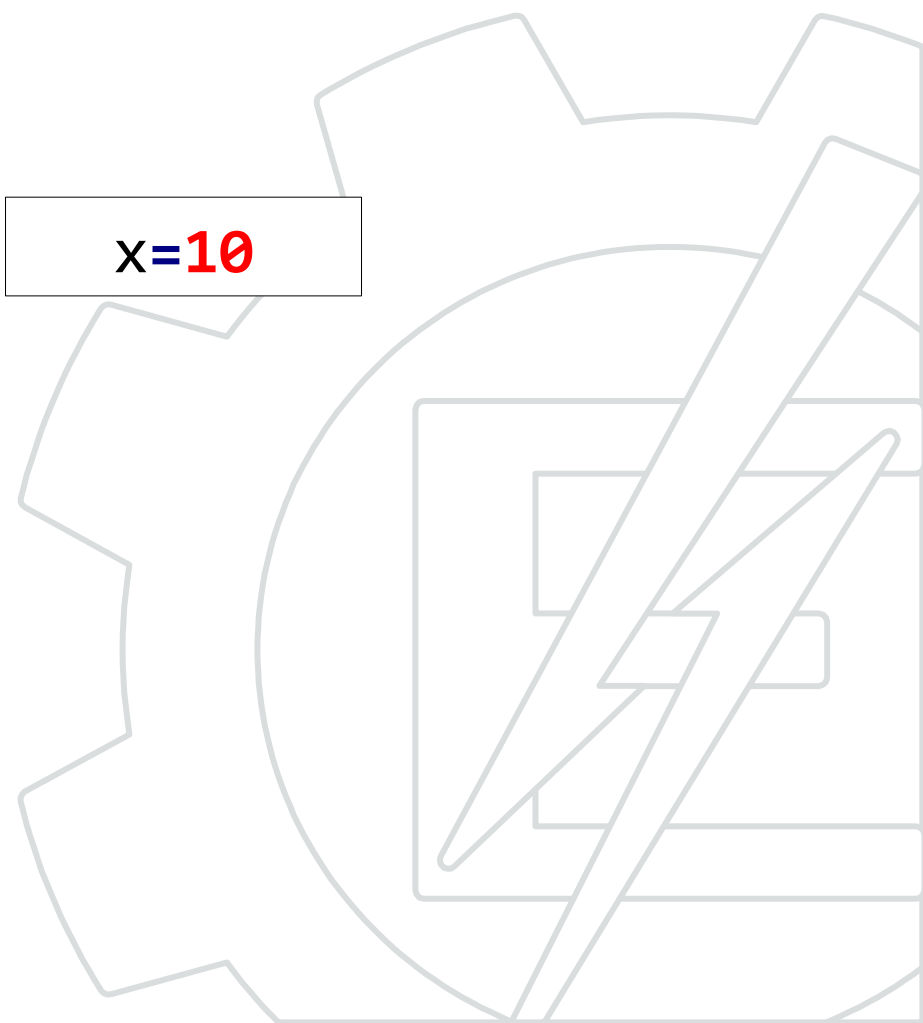
    return;
}
```

Função

```
void inc(int* a){
    (*a) += 1;
    return;
}
```

Variáveis

x=10



Passagem de parâmetros por referência

Rotina Principal

```
void main(void)
{
    int x = 10;

    inc(&x);

    return;
}
```

Função

```
void inc(int* a){
    (*a) += 1;
    return;
}
```

Variáveis

x=10
a = &x

a)



Passagem de parâmetros por referência

Rotina Principal

```
void main(void)
{
    int x = 10;

    inc(&x);

    return;
}
```

Função

```
void inc(int* a){
    (*a) += 1;
    return;
}
```

Variáveis

x=11
a = &x

*

Passagem de parâmetros por referência

Rotina Principal

```
void main(void)
{
    int x = 10;

    inc(&x);

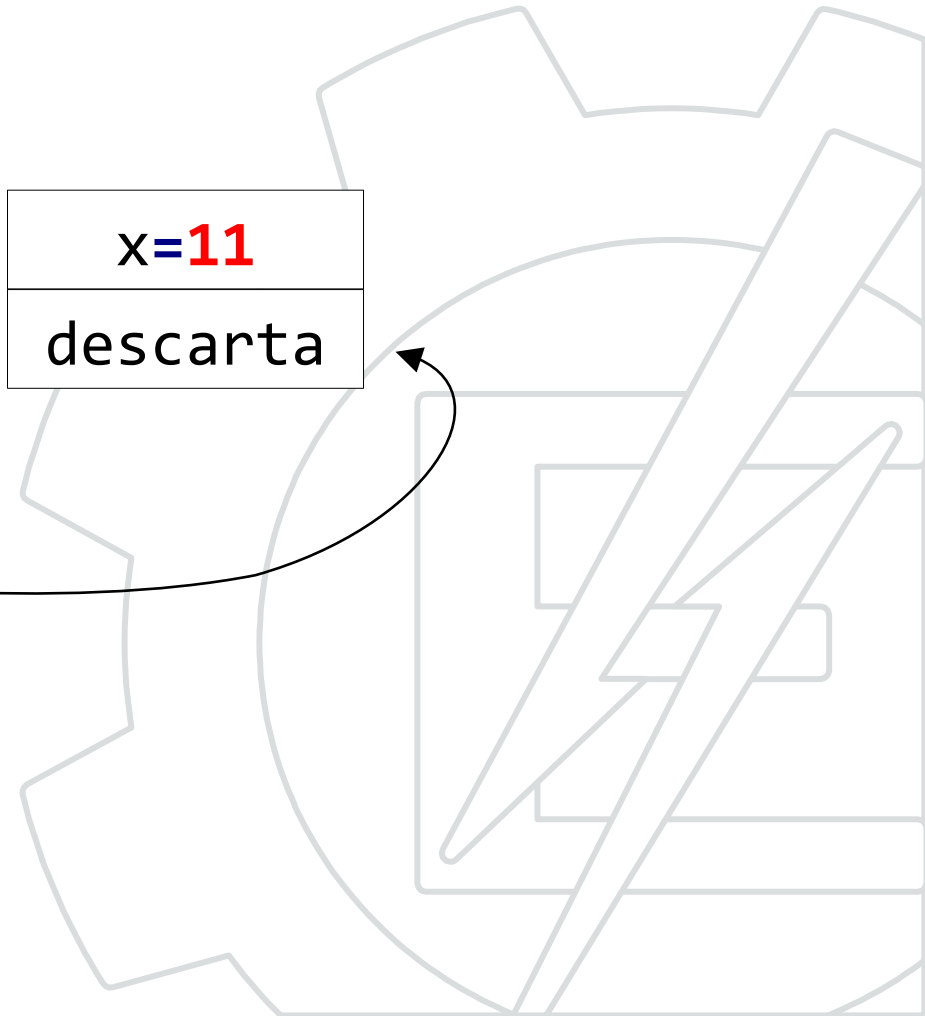
    return;
}
```

Função

```
void inc(int* a){
    (*a) += 1;
    return;
}
```

Variáveis

x=11
descarta

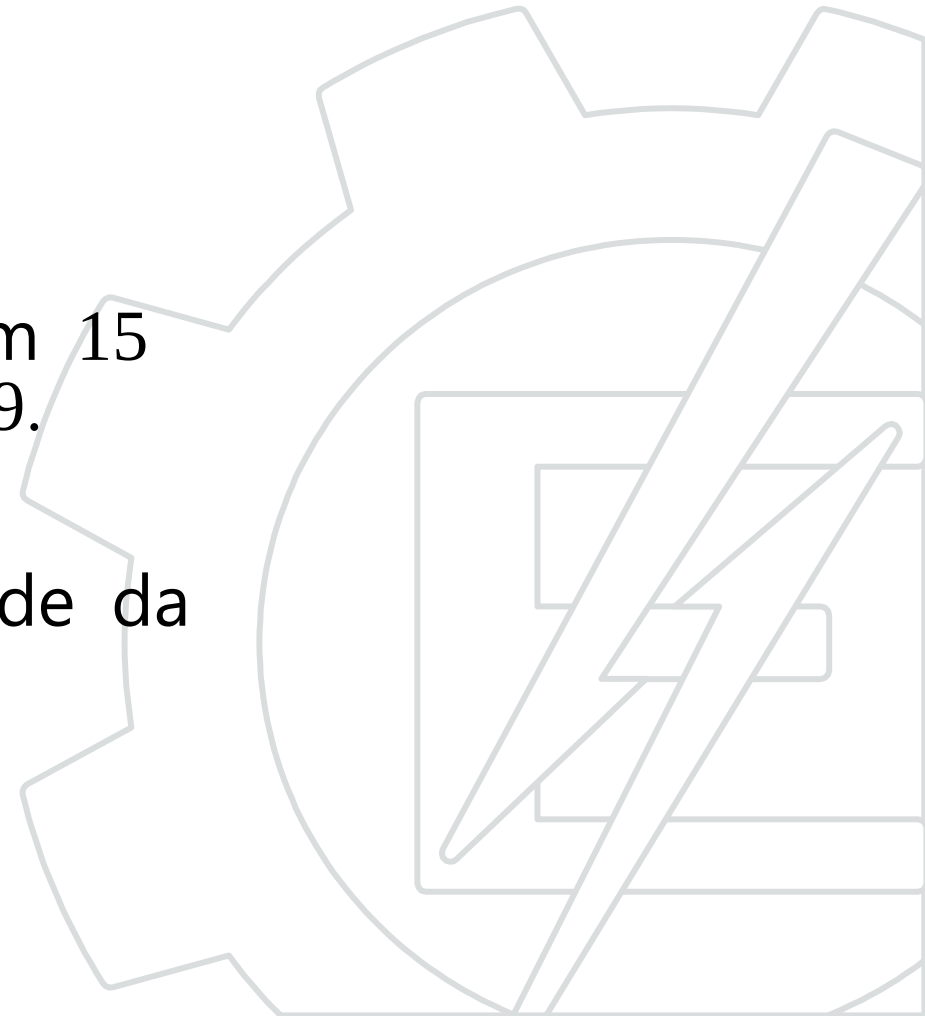


Bibliotecas



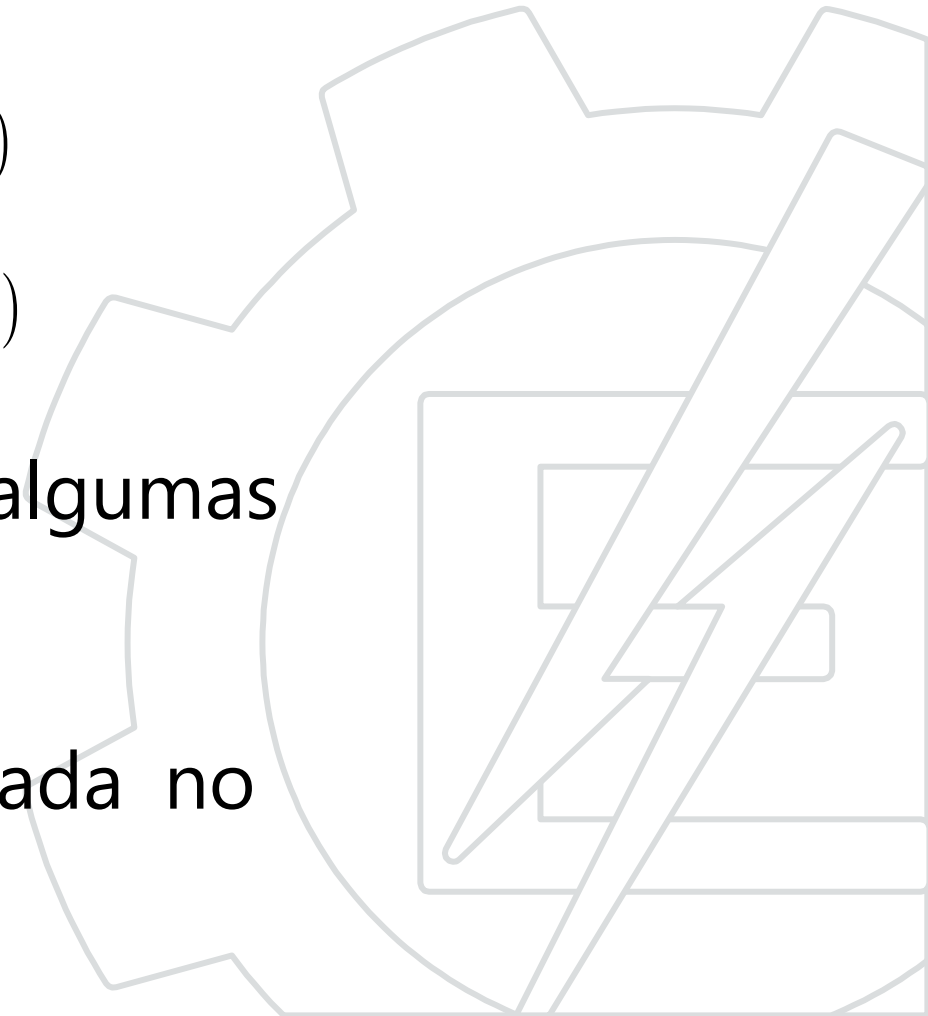
Bibliotecas

- Uma biblioteca é um conjunto de funcionalidades.
- Em geral são separadas por tipo.
- Na primeira versão da linguagem C haviam 15 bibliotecas padronizadas atualmente são 29.
- A disponibilidade destas bibliotecas depende da versão do compilador que está em uso.



Bibliotecas

- A biblioteca é composta de dois arquivos:
 - Um arquivo de cabeçalho, ou header (.h)
 - Um arquivo de código, com extensão (.c)
- O arquivo .h apresenta as funções e algumas definições.
- A implementação das funções é realizada no arquivo .c



Arquivos .c e .h

- Arquivo de código (code)
 - terminado com a extensão .c
 - contém a implementação do código.
 - é compilado gerando um arquivo .o
- Arquivo de cabeçalho (header)
 - terminado com a extensão .h
 - contém apenas “defines” e protótipos.
 - não é compilado.



Exemplo de arquivo .c

```
char digito; //variável global que pode ser modificada pelas funções
```

```
void MudaDigito(char val){  
    digito = val;  
}
```

```
char LerDigito(void){  
    return digito;  
}
```

```
void InicializaDisplays(void){  
    ...  
}
```

```
void AtualizaDisplay(void){ // Função de uso interno  
    ...  
}
```



Exemplo de arquivo .h

```
#ifndef VAR_H
#define VAR_H
    void MudaDigito(char val);
    char LerDigito(void);
    void InicializaDisplays(void);
#endif
```

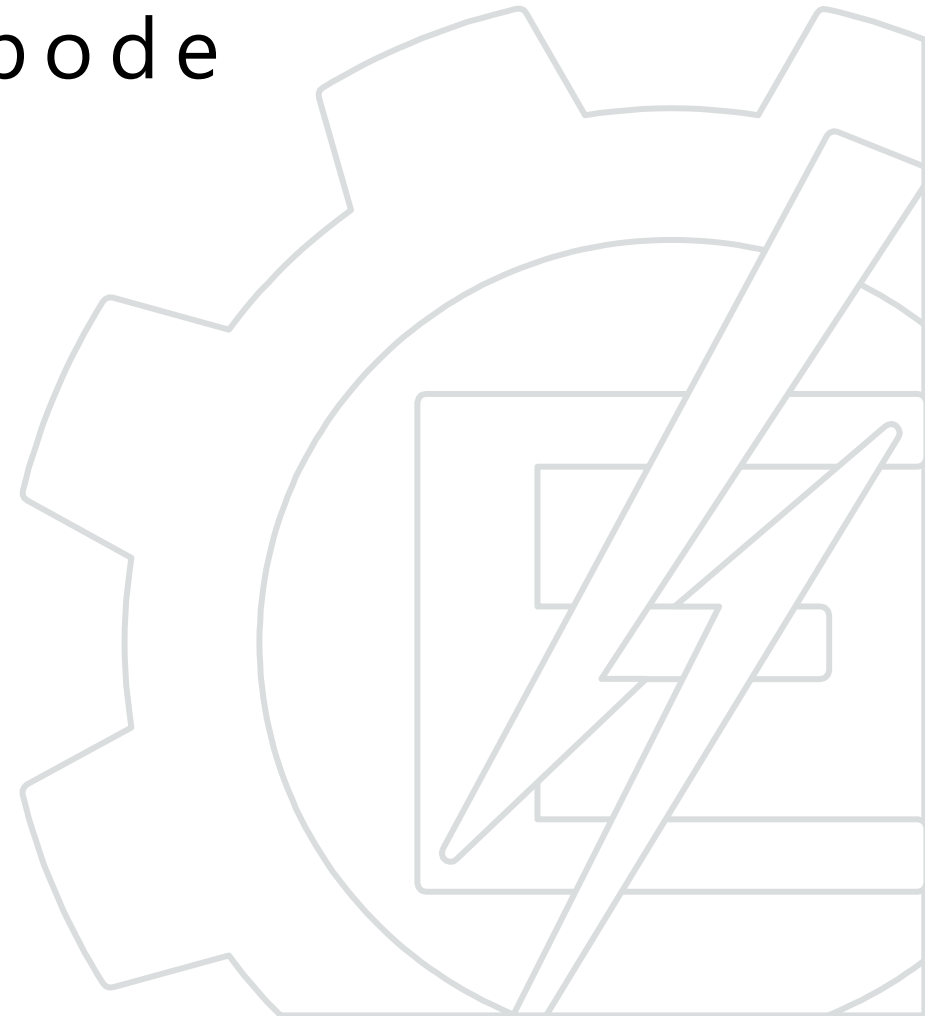
- A função AtualizaDisplay() não é declarada no .h pois só é usada pelas funções internas.
- A variável "digito" só pode ser lida ou gravada pelas funções da biblioteca.

Referência circular

- Quando duas bibliotecas são referenciadas mutuamente pode ocorrer referência circular.

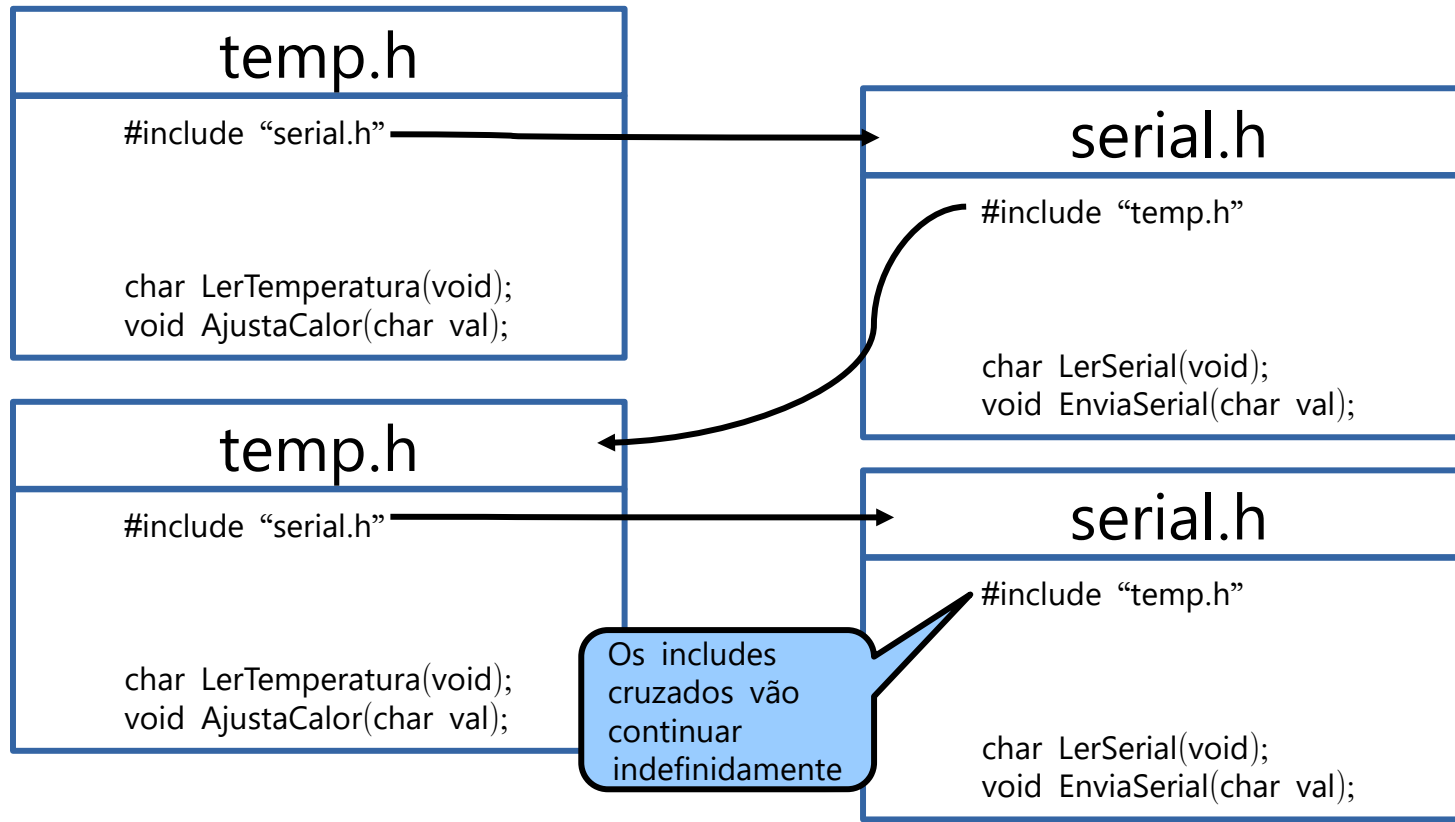
```
//funções da biblioteca de temp  
#include "serial.h"  
char LerTemperatura(void);  
void AjustaCalor(char val);
```

```
//funções da biblioteca serial  
#include "temp.h"  
char LerSerial(void);  
void EnviaSerial(char val);
```



Referência circular

- Elas são incluídas infinitamente:



Referência circular

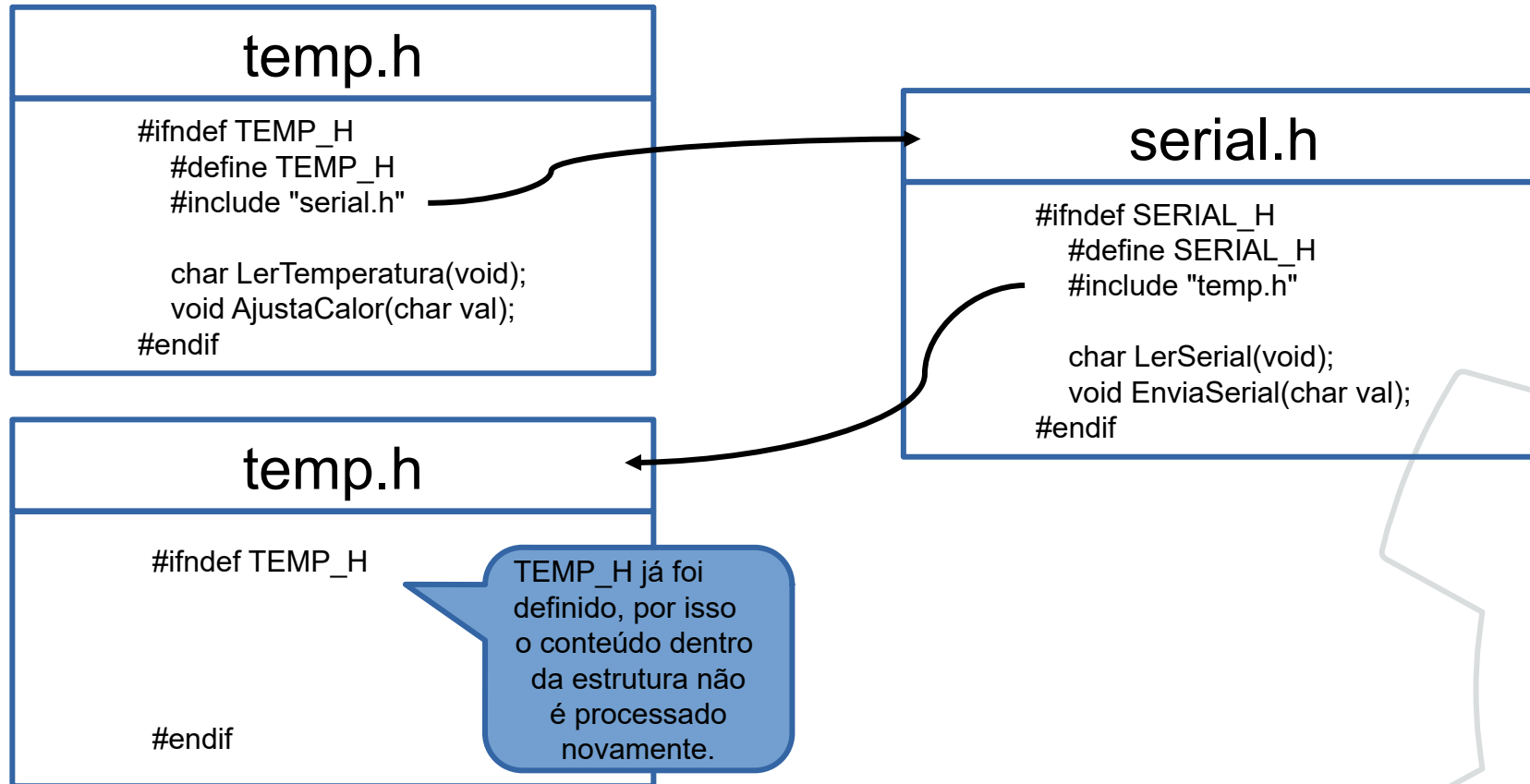
- Solução: criar uma estrutura de controle para pré compilação.

```
#ifndef TAG_CONTROLE
#define TAG_CONTROLE
    //todo o conteúdo do arquivo vem aqui.

#endif //TAG_CONTROLE
```



Referência circular



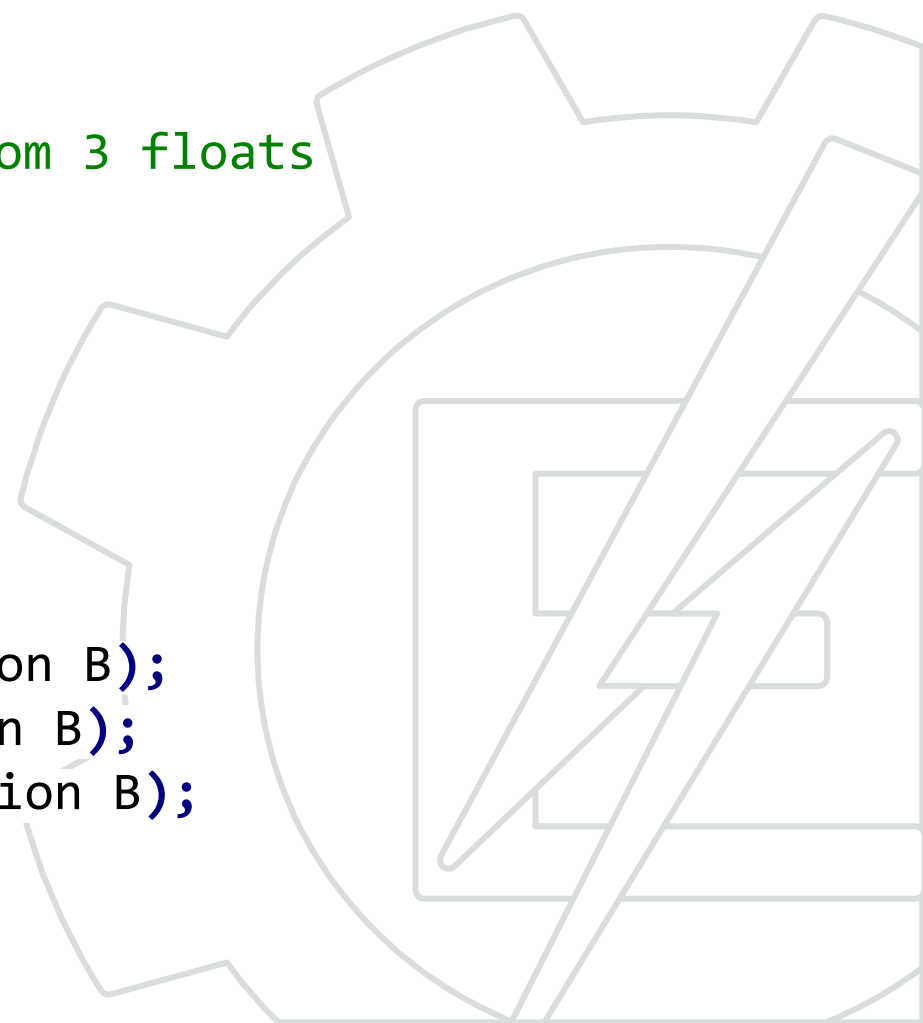
Exemplo header

```
//header: dist3d.h
#ifndef DIST3D_H
    #define DIST3D_H

    //define o tipo position como uma struct com 3 floats
    typedef struct{
        float x;
        float y;
        float z;
    } position;

    //cálculo de distâncias entre 2 pontos
    float CartesianDistance(position A, position B);
    float ManhattanDistance(position A, position B);
    float AltitudeDifference(position A, position B);

#endif
```



Exemplo código (arquivo .c)

```
#include "dist3d.h"
```

```
//Calcular o quadrado de um número
```

```
float quad (float val){ return val * val; }
```

```
//retorna o módulo
```

```
float mod(float a){      return (a>0 ? a: -a);}
```

```
//Calcular a raiz quadrada de um número, adaptado do Quake Fast InvSqrt()
```

```
float sqrt( float number ){  
    long i; float x2, y; const float threehalfs = 1.5F;  
    x2 = number * 0.5F;          y = number;  
    i = * ( long * ) &y; i = 0x5f3759df - ( i >> 1 );  
    y = * ( float * ) &i; y = y * (threehalfs - (x2 * y * y));  
    return (1/y);  
}
```

```
float CartesianDistance(position A, position B){  
    return sqrt(quad(A.x-B.x) + quad(A.y-B.y) + quad(A.z-B.z));  
}
```

```
float ManhattanDistance(position A, position B){  
    return mod(A.x - B.x) + mod(A.y - B.y) + mod(A.z - B.z);  
}
```

```
float AltitudeDifference(position A, position B){  
    return mod(A.z - B.z);  
}
```

Planejando o software embarcado

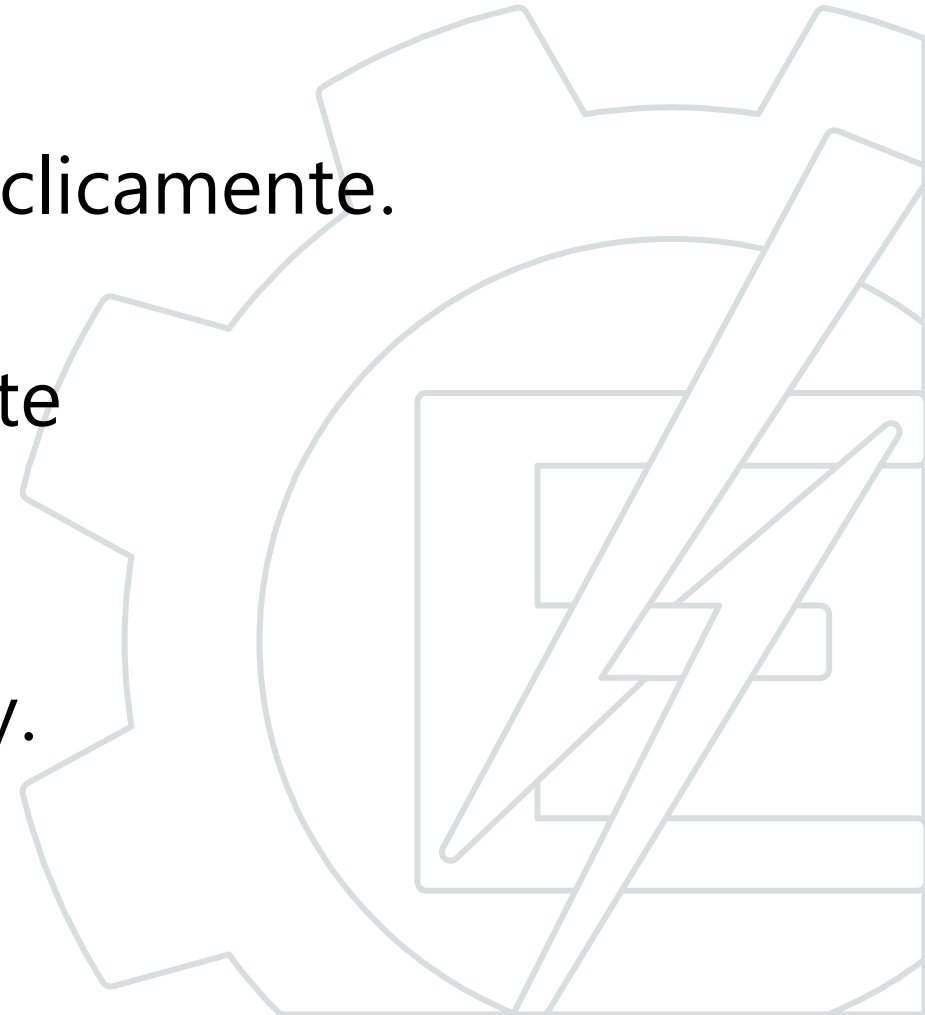


Estrutura básica



Estrutura básica

- 1) Todos os periféricos importantes devem ser inicializados.
- 2) O programa principal deve rodar ciclicamente.
- 3) Nenhuma rotina deve ser bloqueante (aguardando alguma coisa).
- 4) Tome cuidado com rotinas de delay.



```
//includes
#include <stdio.h>
#include "lcd.h"

//defines
#define MAX_VAL 42

//variáveis globais
unsigned char max;

//protótipos de funções
void reinicializa_variaveis(void);

void main(void){
    //variáveis locais
    /*inicialização sistema*/

    for(;;){/*loop infinito*/
        //execução atividade 1
        //....
        //execução atividade N
    }
}

//funções
void reinicializa_variaveis(void){
    max = MAX_VAL;
}
```

Estrutura básica

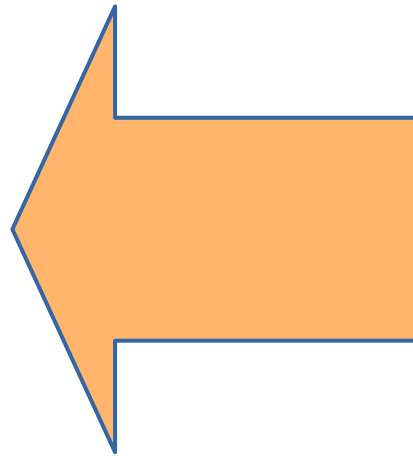


Imagem: wdrfree.com

Isso é muito importante pessoal

Até a próxima!

